



Assessment Title

Project(Individual)

Assessment Type

Uncontrolled

Individual

Not must-pass

Due Date

8th January 2026

Course Code

IT9002

Course Title

Natural Language Processing

Internal Moderator's

Dr. Abdelhameed Fawzy

External Examiner's

Dr.

Instructions:

1. This cover sheet must be completed (section in blue below) and attached to your assessment before submission in hard copy/soft copy.
2. The time allowed for this assessment is XXX minutes/hours/days.
3. This assessment carries XXX marks distributed to a total of XXX questions assessing CILO X and CILO X.
4. The materials allowed for use in this assessment are XXX, XXX, and XXX.
5. The use of generative AI tools is strictly prohibited.
6. References consulted (if any) must be properly acknowledged and cited.
7. The assessment has a total of XXX pages.

Learner ID

202508958

Date Submitted

8th January 2025

Learner Name

Fatema Husain Hasan

Programme Title

MSc in AI

Programme Code

IT9002

Lecturer's Name

Sini Raj Pulari

By submitting this assessment for marking, I affirm that this assessment is my own work.

Do not write beyond this line. For assessor use only.

Learner Signature

Assessor's Name

Marking Date

Marks Obtained

Comments:

Table of Contents

Task 1 - Problem Statement Formulation and Definition	5
1.1 Introduction.....	5
1.2 Motivation.....	5
1.3 Problem Statement.....	6
1.4 Project Definition.....	6
1.5 Objectives	6
1.6 Expected Outcomes	6
Task 2 - Selection of an Appropriate Data Set (Data Collection).....	7
2.1 Dataset Overview	7
2.2 Dataset Source	7
2.3 Dataset Selection Justification.....	7
2.4 Dataset Structure and Shape	8
2.5 Description of Key Attributes	9
2.6 Label Identification and Transformation	9
2.7 Exploratory Data Analysis and Visualization	10
Task 3 – Text Preprocessing	11
3.1 Introduction to text Preprocessing.....	11
3.2 Lowercasing	11
3.3 Emoji Removal.....	12
3.4 Punctuation and Numbers Removal.....	12
3.5 Tokenization.....	13
3.6 Stopword Removal.....	13
3.7 Stemming	14
3.8 Lemmatization.....	14
3.9 Final Preprocessed Text.....	15
Task 4 – Text Representation	15
4.1 Introduction to Text Representation.....	15
4.2 Bag of Words (BoW)	15

4.3 N-grams Representation	18
4.4 Part-of-Speech (POS) Tagging.....	19
4.5 Named Entity Recognition (NER)	20
Task 5 - Text Classification / Prediction	20
5.1 Task 5 Introduction.....	20
5.2 Sentiment Label Creation	21
5.3 Train and Test Data Split.....	21
5.4 Feature Extraction Using TF-IDF	22
5.5 Model Selection and Justification	22
5.5.1 Multinomial Naive Bayes.....	22
5.5.2 Logistic Regression	23
5.6 Model Training and Prediction	23
Task 6 - Evaluation, Inferences, Recommendation and Reflection.....	24
6.1 Task 6 Introduction.....	24
6.2 Evaluation Metrics	24
6.3 Model Performance Evaluation	25
6.4 Model Comparison.....	25
6.7 Project Enhancements	27
6.8 Learning Reflection	28
References	29

Table of Figures

Figure 1 - Import Libraries.....	8
Figure 2 - Presenting first 5 rows in the dataset.....	8
Figure 3 - Presenting the dataset shape and columns.....	8
Figure 4 - Labelling the data into sentiment classes	9
Figure 5 - Labelling the data visualization	10
Figure 6 – Lowercasing	11
Figure 7 - Emoji Removal	12
Figure 8 - Punctuation and Numbers Removal.....	12
Figure 9 - Tokenization.....	13
Figure 10 - Stopword Removal.....	13
Figure 11 - Stemming	14
Figure 12 - Lemmatization.....	14
Figure 13 - Final Preprocessed Text.....	15
Figure 14 - Bag of Words.....	16
Figure 15 - Bag of Words Visualization	17
Figure 16 - Top 20 Words in BoW Visualization	17
Figure 17 - N-gram.....	18
Figure 18 - Top 20 words Bigrams.....	18
Figure 19 - POS Tagging	19
Figure 20 - Top 10 POS Visualization	19
Figure 21 - NER with example	20
Figure 22 - Sentiment Label Creation and testing.....	21
Figure 23 - Train and Test Data Split.....	21
Figure 24 - TF-IDF	22
Figure 25 - Multinomial Naive Bayes.....	23
Figure 26 - Logistic Regression	23
Figure 27 - Evaluation Metrics	24
Figure 28 - Confusion Matrix Naive Bayes.....	26
Figure 29 - Confusion Matrix Logistic Regression	27

Task 1 - Problem Statement Formulation and Definition

1.1 Introduction

The fast increase in e-commerce sites is making customers rely increasingly on online reviews as a means of sharing their minds and experiences with products and services. These ratings play an important role in the buying decisions of the consumers and provide companies with valuable data. The examination of customer reviews by a human, however, is a time-consuming and impracticable undertaking due to the colossal amount of the textual data generated by users. Natural language processing, or NLP, provides effective methods of automatic analysis and extraction of significant information of unstructured text data. NLP based text categorization methods can help businesses understand the urges of the clients, identify potential issues, and improve the decision-making process.

1.2 Motivation

The primary objective of this project is to explore the possible application of the NLP techniques that I have studied in this course in deriving helpful information about the actual business data. Online shopping websites such as Amazon are the recipients of thousands of evaluations every day which reflect consumer happiness, dissatisfaction, and neutral opinions. By understanding the sentiment of consumers on a large scale, the following things can be done by businesses:

- Monitor the level of customer satisfaction.
- Identify negative remarks in good time.
- Increase customer care and quality of products.
- Enhance the overall user experience.

1.3 Problem Statement

To design and implement an automatic Natural Language Processing based program that assesses Amazon customer reviews and separates them into three sentiment groups (positive, neutral and negative) based on the text of the reviews.

1.4 Project Definition

The objective of this project is to utilize natural language processing (NLP) strategies to pre-process customer review data, turn it into the relevant numerical form, and develop machine learning models, which can be used to categorize sentiment. The project consists of numerous stages, including model training, evaluation, text representation, and data preparation. The system anticipates sentiment categories of unseen reviews through supervised learning methods to determine patterns in labeled review data.

1.5 Objectives

The key objectives of the project are as follows:

- To apply NLP techniques to unprocessed review data of customers. In order to employ appropriate methods of feature extraction to quantify the textual information.
- Principles of using and comparing various text categorization models.
- To evaluate the performance of the model using traditional categorization measures.
- To extract useful data from sentiment analysis of customers.

1.6 Expected Outcomes

The expected outcomes of this project at the end of its implementation are as follows:

- A preprocessed and cleansed Amazon reviews dataset.
- Representations that are effective with machine learning models.
- Models of classification that can predict consumer mood using training.
- Performance of models in comparison.
- Business-relevant insights obtained through sentiment analysis.

Task 2 - Selection of an Appropriate Data Set (Data Collection)

2.1 Dataset Overview

This study was conducted using an Amazon customer reviews dataset, which is updated daily to perform text-based sentiment categorization. The data set is also perfectly suited to Natural Language Processing tasks using sentiment analysis as it contains real user-generated ratings collected on an e-commerce platform. The collection contains textual customer reviews and metadata, including ratings and timestamps. It becomes feasible to apply the supervised learning techniques to sentiment prediction.

2.2 Dataset Source

Dataset Name: Amazon Shopping Reviews (Daily Updated)

Source: Kaggle

Domain: E-commerce / Online Retail

The data was acquired in Kaggle due to the availability of well-documented, publicly available datasets frequently used in machine learning and academic studies (Kumar, 2025). Kaggle datasets are credible and commonly used in research on sentiment analysis using NLP.

2.3 Dataset Selection Justification

This dataset was chosen because of the following factors:

- It possesses a large number of verifiable customer reviews that reflect the opinion of real users.
- The text is not structured and, therefore, it is ideal to apply NLP preprocessing and feature extraction to it.
- The data represents a practical business scenario where automated sentiment analysis can be used to provide valuable information to support decision-making.

2.4 Dataset Structure and Shape

The structure of the dataset was examined to know the number of records and attributes on it after loading it.

Rows: Present testimonials of different clients.

Columns: Include review id, username, content, score, thumbs up count, review Created Version, at (timestamp) app Version.

```
#Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import re
import nltk
import spacy

from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from nltk import pos_tag

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.feature_extraction.text import CountVecorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.decomposition import PCA
from wordcloud import WordCloud
from collections import Counter
from spacy import displacy

nltk.download('stopwords')
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
```

Figure 1 - Import Libraries

```
# Show the first 5 rows of the dataset
df.head()
```

	reviewId	userName	content	score	thumbsUpCount	reviewCreatedVersion	at	appVersion
0	b00cac20-ed09-4368-8e3a-13b9b181d9a5	Michael Simmons	awesome 🍌	5	0	30.22.0.100	2025-12-20 10:35:11	30.22.0.100
1	eba2e27b-cf41-4c64-a0da-b7819f887e40	Rahamtullah Khan	bad	1	0	30.22.2.100	2025-12-20 10:27:54	30.22.2.100
2	664ca9b7-93ae-438a-9102-a2da47ba31c1	Alda Wright	Convenience, Convenience, Convenience!!!	5	0	30.22.0.100	2025-12-20 09:47:31	30.22.0.100
3	1eceb57e-21e8-4b53-af01-a5098127d3ec	Zak Cullen	Terrible app and super difficult with changing...	2	0	30.22.2.100	2025-12-20 09:30:19	30.22.2.100
4	457773fe-ce0f-4849-b460-be2b02c7cda0	Luna K	nifty app	5	0	30.22.0.100	2025-12-20 09:17:31	30.22.0.100

Next steps: [Generate code with df](#) [New interactive sheet](#)

Figure 2 - Presenting first 5 rows in the dataset

```
# Show Dataset shape
df.shape
```

(82449, 8)

```
# Show dataset columns
df.columns
```

Index(['reviewId', 'userName', 'content', 'score', 'thumbsUpCount', 'reviewCreatedVersion', 'at', 'appVersion'], dtype='object')
--

Figure 3 - Presenting the dataset shape and columns

2.5 Description of Key Attributes

The attributes that are most relevant to this project are as follows:

Content: Customer written textual reviews are included. This column forms the primary input of NLP processing.

Score: Refers to the rating of the customer number. Based on this column, sentiment labels are obtained.

Other columns (**reviewId**, **Username** and **timestamp**) do provide more metadata, though not directly assist in model training.

2.6 Label Identification and Transformation

The data set originally had numerical ratings that measured the level of low and high. These ratings were transformed into three sentiment categories to be used in sentiment classification by using the if condition as seen in figure 4 and classified as below:

- **Negative:** Poor ratings (1-2 Stars)
- **Neutral:** Medium ratings (3 Stars)
- **Positive:** Excellent ratings (4-5 Stars)

This transformation is in line with the conventional practices of sentiment analysis research and allows application of multi-class classification algorithms.

```
# Label the data into sentiment classes
def sentiment_label(rating):
    if rating <= 2:
        return 'Negative'
    elif rating == 3:
        return 'Neutral'
    else:
        return 'Positive'

df['sentiment'] = df['score'].apply(sentiment_label)
```

Figure 4 - Labelling the data into sentiment classes

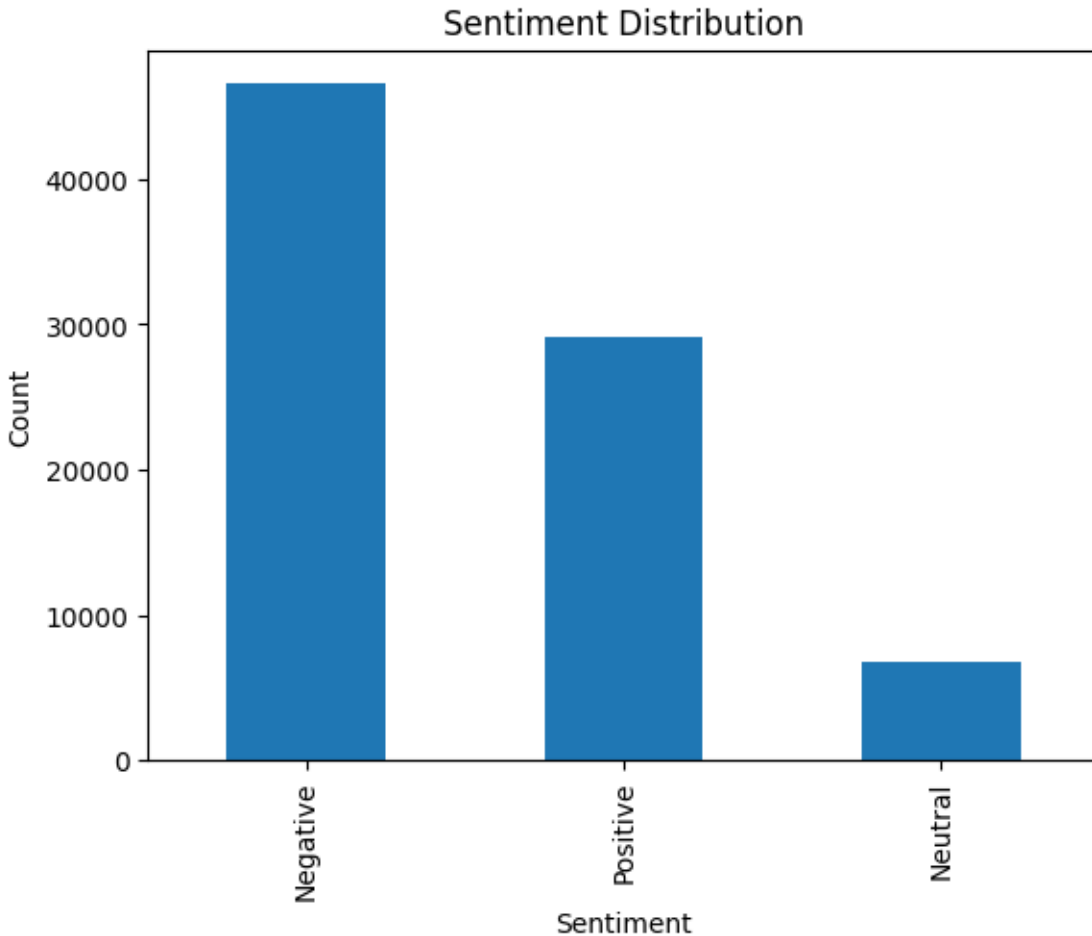


Figure 5 - Labelling the data visualization

2.7 Exploratory Data Analysis and Visualization

The exploratory analysis was conducted to get a better insight into the data: The distribution of labels of emotions was presented in the form of bar charts. Sample reviews were assessed to understand the length of the text, the language usage and the type of writing. Since it is typical of e-commerce websites where satisfied customers are better positioned to leave feedback, the graph indicated that the proportion of positive ratings was greater in the dataset compared to the rate of neutral and negative ratings.

Task 3 – Text Preprocessing

3.1 Introduction to text Preprocessing

Text preprocessing is a key step in natural language processing because text raw data often includes noise (Deepanshi, 2025) such as irregular cases, punctuation, emojis, stopwords and repeated words. Failure to deal with such noise can affect the performance of machine learning models. The text preprocessing produced by this project aims at transforming unstructured unprocessed customer review of Amazon into a format that is appropriate to use in sentiment classification and feature extraction.

3.2 Lowercasing

In this task lowercasing is applied to convert all the letters in the text to a lower case by using NLTK tokenizer as shown in the figure below (figure 6). This ensures that when words of identical meaning but different letter casing like “Bad” and “bad” are analyzed they are treated as the same. The use of lowercase helps to improve consistency across the dataset and reduce the size of vocabulary.

```
# Lowercasing
df['lower_text'] = df['content'].str.lower()

df[['content', 'lower_text']].head()
## we can see that the new text all lowercased
```

	content	lower_text
0	awesome 🙌	awesome 🙌
1	bad	bad
2	Convenience, Convenience, Convenience!!!	convenience, convenience, convenience!!!
3	Terrible app and super difficult with changing...	terrible app and super difficult with changing...
4	nifty app	nifty app

Figure 6 – Lowercasing

3.3 Emoji Removal

Customer reviews often include the use of emojis to express emotions. At times, emojis can convey emotion, though in this project, they were discarded to simplify textual analysis and get down to sentiment patterns by word as shown in the figure below along with the results (figure 7). Textual data can be made more understandable by emoji removal to traditional machine learning models and can be used to eliminate non-textual noise.

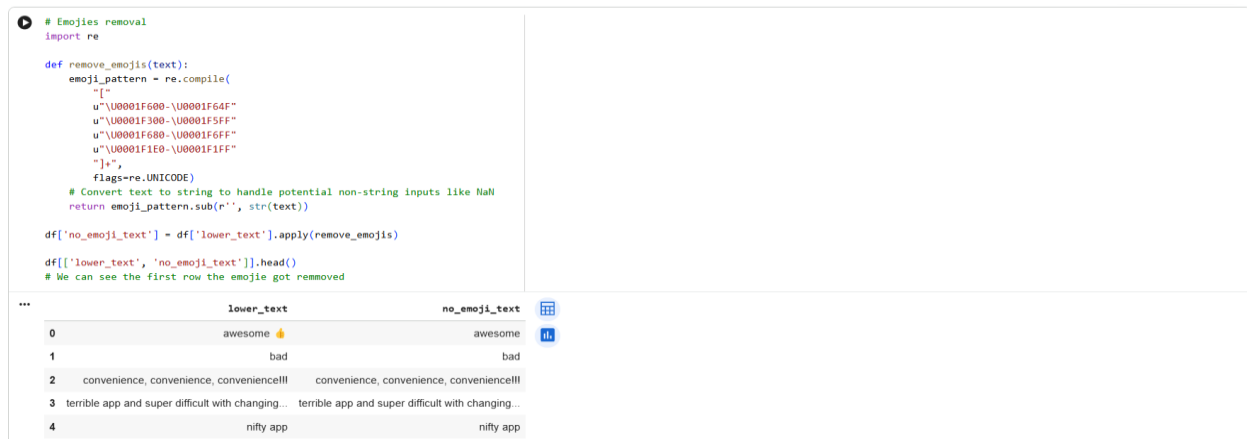


Figure 7 - Emoji Removal

3.4 Punctuation and Numbers Removal

Overall, punctuation and number characters do not give helpful data on emotion classification. To retain only the alphabetic characters, it became necessary to remove them. Besides the fact that the dataset must include clear textual data that can be tokenized and used to extract features, this step will reduce the amount of noise as shown in the figure below with the results before and after (figure 8).

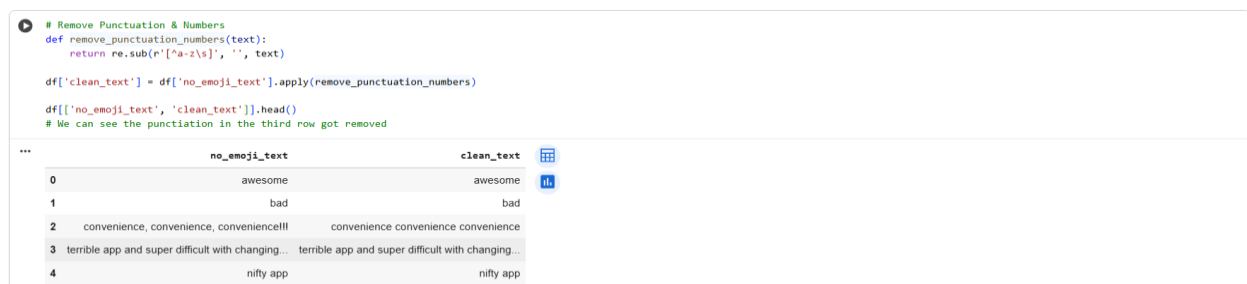


Figure 8 - Punctuation and Numbers Removal

3.5 Tokenization

In this task we used the tokenization process of breaking down text into separate words, or tokens. Rather than text being analyzed as a continuous string, the result allows the NLP pipeline to analyze text on the word level. In this project, the tokenization method was employed to divide the customer reviews into individual word tokens which form the foundation of other preprocessing tasks such as lemmatization, stemming and removal of stopwords as shown in the figure below (figure 9).

```
# Tokenization
from nltk.tokenize import word_tokenize

df['tokens'] = df['clean_text'].apply(word_tokenize)

df[['clean_text', 'tokens']].head()
```

	clean_text	tokens
0	awesome	[awesome]
1	bad	[bad]
2	convenience convenience convenience	[convenience, convenience, convenience]
3	terrible app and super difficult with changing...	[terrible, app, and, super, difficult, with, c...
4	nifty app	[nifty, app]

Figure 9 - Tokenization

3.6 Stopword Removal

Stopwords Words such as “is, the, and” and that are frequently used, but usually do not provide much information in terms of sentiment categorization tasks. In this task removing stopwords by using NLTK tool will help the models to focus on more informative words that can help predict the sentiment in addition to dimensionality reduction as shown in the figure below (figure 10).

```
# Stopword Removal
from nltk.corpus import stopwords

stop_words = set(stopwords.words("english"))

def remove_stopwords(text):
    tokens = word_tokenize(text)
    filtered_tokens = [word for word in tokens if word not in stop_words]
    return filtered_tokens

df['tokens_no_stopwords'] = df['tokens'].apply(
    lambda tokens: [word for word in tokens if word not in stop_words])

df[['tokens', 'tokens_no_stopwords']].head()
```

	tokens	tokens_no_stopwords
0	[awesome]	[awesome]
1	[bad]	[bad]
2	[convenience, convenience, convenience]	[convenience, convenience, convenience]
3	[terrible, app, and, super, difficult, with, c...	[terrible, app, super, difficult, changing, ad...
4	[nifty, app]	[nifty, app]

Figure 10 - Stopword Removal

3.7 Stemming

In this step stemming is used to remove suffixes from the simplest form of words however sometimes it might create words with no meaning as shown in the figure below (figure 11) . An example is that the words such as liked, liking and likes are simplified into like. In such a way, the size of vocabulary is reduced and similar words in the form are collocated.

```
# Stemming
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

df['stemmed_tokens'] = df['tokens_no_stopwords'].apply(
    lambda tokens: [stemmer.stem(word) for word in tokens]
)

df[['tokens_no_stopwords', 'stemmed_tokens']].head()
```

	tokens_no_stopwords	stemmed_tokens
0	[awesome]	[awesom]
1	[bad]	[bad]
2	[convenience, convenience, convenience]	[conveni, conveni, conveni]
3	[terrible, app, super, difficult, changing, ad...]	[terribl, app, super, difficult, chang, addres...]
4	[nifty, app]	[nifti, app]

Figure 11 - Stemming

3.8 Lemmatization

Unlike stemming, Lemmatization gives meaning to meaningful words and increases interpretability. In this task we employed Lemmatization to get standard word forms without sacrificing the semantic meaning when stopwords were removed.

```
# Lemmatization
lemmatizer = WordNetLemmatizer()

df['lemmatized_tokens'] = df['stemmed_tokens'].apply(
    lambda tokens: [lemmatizer.lemmatize(word) for word in tokens]
)

df[['stemmed_tokens', 'lemmatized_tokens']].head()
```

	stemmed_tokens	lemmatized_tokens
0	[awesom]	[awesome]
1	[bad]	[bad]
2	[conveni, conveni, conveni]	[convenience, convenience, convenience]
3	[terribl, app, super, difficult, chang, addres...]	[terrible, app, super, difficult, changing, address...]
4	[nifti, app]	[nifty, app]

Figure 12 - Lemmatization

3.9 Final Preprocessed Text

After all preprocessing processes had been completed, the cleaned tokens were recreated into one text string in case of each review. This end result of preprocessed text is used as input in later tasks in text representation and classification models. The preprocessing pipeline highly enhances the effectiveness of the feature extraction and machine learning models, reduces noise, and enhances the quality of data.

```
# Final text
df['final_text'] = df['lemmatized_tokens'].apply(lambda x: ' '.join(x))
df[['content', 'final_text']].head()
```

	content	final_text
0	awesome 🙌	awesom
1	bad	bad
2	Convenience, Convenience, Convenience!!!	conveni conveni conveni
3	Terrible app and super difficult with changing...	terribl app super difficult chang address card...
4	nifty app	nifti app

Figure 13 - Final Preprocessed Text

The figure above represents the text before and after the text preprocessing. We can see that emojis and punctuation got removed. In addition, lower casing and other preprocessing methods were applied successfully.

Task 4 – Text Representation

4.1 Introduction to Text Representation

The second step in the NLP pipeline is text representation involving representation of the text that has been preprocessed. Text representation strategies gather language patterns, word usage and context in a way that is suitable for model training. To ensure that the syntactic and semantic attributes of customer reviews were effectively captured, the following types of text representations were employed in the study.

4.2 Bag of Words (BoW)

The Bag of Words model does not pay attention to word order and grammar instead of text representation that is achieved through the computation of the frequency of each word in the

sample. Each review is turned into a vector, and each dimension is a specific term and its frequency. As a simple and practical baseline form of sentiment categorization, Bag of Words does not consider the word sequence. In this undertaking, the BoW has applied to locate frequent expressions used by customers in their reviews and understand dominant issues related to consumer satisfaction and product experience. Some of the frequently referred qualities that were highlighted in a graphic of the most used terms included product quality, performance and usability.

```
# Bag of Words
from sklearn.feature_extraction.text import CountVectorizer

bow_vectorizer = CountVectorizer(max_features=3000)
X_bow = bow_vectorizer.fit_transform(df['final_text'])

X_bow.shape

... (82449, 3000)

# Get word frequencies (summary counts across all reviews)
word_counts = np.asarray(X_bow.sum(axis=0)).flatten()
words = bow_vectorizer.get_feature_names_out()

bow_frequencies = dict(zip(words, word_counts))

# WordCloud from BoW frequencies
wordcloud = WordCloud(background_color='white', width=800, height=400)\
    .generate_from_frequencies(bow_frequencies)

plt.figure(figsize=(12, 6))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title("Word Cloud of Bag of Words (Top Features)")
plt.show()
```

Figure 14 - Bag of Words

[illegible]

Figure 15 - Bag of Words Visualization

Words	Frequency
app	41000
amazon	38000
order	18500
get	17000
use	17000
item	15000
time	15000
shop	12500
dont	10500
cant	10500
prime	10500
servic	10500
deliveri	10000
updat	10000
custom	9500
day	9500
like	8500
product	8500
work	8500
even	8500

Figure 16 - Top 20 Words in BoW Visualization

4.3 N-grams Representation

Bag of Words is used to estimate how often words are used but it does not consider the relationship between words that follow each other. To overcome this limitation, n-gram representation was used. N-grams are sequences of n consecutive words. This research involved both unigrams and bigrams. This method can capture sentiment-bearing phrases such as not good, very bad, and high quality, which are often necessary in sentiment analysis. N-grams enhance the classification and ability of the model to extract contextual patterns.

```
# N-Gram
ngram_vectorizer = CountVectorizer(ngram_range=(1,2), max_features=4000)
X_ngrams = ngram_vectorizer.fit_transform(df['final_text'])

X_ngrams.shape
... (82449, 4000)

# Visualize Bigram
ngram_counts = np.asarray(X_ngrams.sum(axis=0)).ravel()
ngrams = np.array(ngram_vectorizer.get_feature_names_out())

# filter only bigrams (contain space)
bigram_mask = np.char.find(ngrams.astype(str), " ") >= 0
bigrams = ngrams[bigram_mask]
bigram_counts = ngram_counts[bigram_mask]

top_n = 20
top_idx = bigram_counts.argsort()[-top_n:][::-1]

plt.figure(figsize=(10,5))
plt.bar(bigrams[top_idx], bigram_counts[top_idx])
plt.xticks(rotation=45, ha='right')
plt.title("Top 20 Bigrams")
plt.xlabel("Bigrams")
plt.ylabel("Frequency")
plt.show()
```

Figure 17 - N-gram

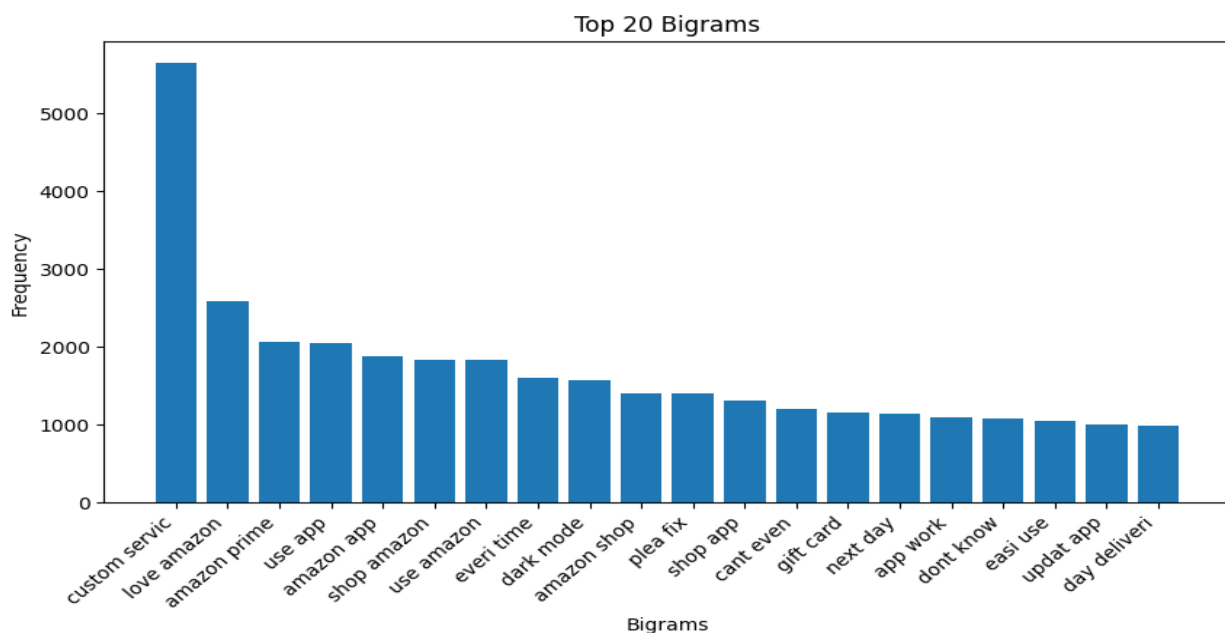


Figure 18 - Top 20 words Bigrams

4.4 Part-of-Speech (POS) Tagging

Part-of-speech tagging is used to assign each word in the text as grammatical label (e.g. nouns, verbs and adjectives). The POS tagging helps to identify word categories that are particularly important to sentiment analysis, as well as provides the information on the syntactic structure of sentences. Strong feeling is common in customer assessment in the use of adjectives and adverbs, e.g. excellent, terrible. The distribution of POS tags may be studied to determine the trends in consumer views and emotions. To demonstrate grammar structure and to determine commonly occurring parts of speech, POS tagging was performed to a subset of reviews.

```
# POS Tagging
import nltk
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')

from nltk import pos_tag
from nltk.tokenize import word_tokenize

# 3 sample reviews for demonstration
sample_texts = df['final_text'].dropna().sample(3, random_state=42).tolist()

for i, text in enumerate(sample_texts, 1):
    tokens = word_tokenize(text)
    tagged = pos_tag(tokens)
    print(f"\n--- Sample Review {i} ---")
    print(tagged[:30]) # show first 30 tags

...

--- Sample Review 1 ---
[('high', 'JJ'), ('price', 'NN')]

--- Sample Review 2 ---
[('freez', 'NN'), ('constantli', 'NN'), ('sinc', 'NN'), ('last', 'JJ'), ('updat', 'JJ'), ('headach', 'NN'), ('plea', 'NN'), ('fix', 'NN')]

--- Sample Review 3 ---
[('instal', 'JJ'), ('app', 'NN'), ('work', 'NN'), ('ok', 'JJ'), ('turn', 'VBP'), ('phone', 'NN'), ('amazon', 'NN'), ('app', 'NN'), ('fulli', 'NN'), ('disappear', 'VBP'), ('reinstal', 'JJ'), ('app', 'NN')]
```

Figure 19 - POS Tagging

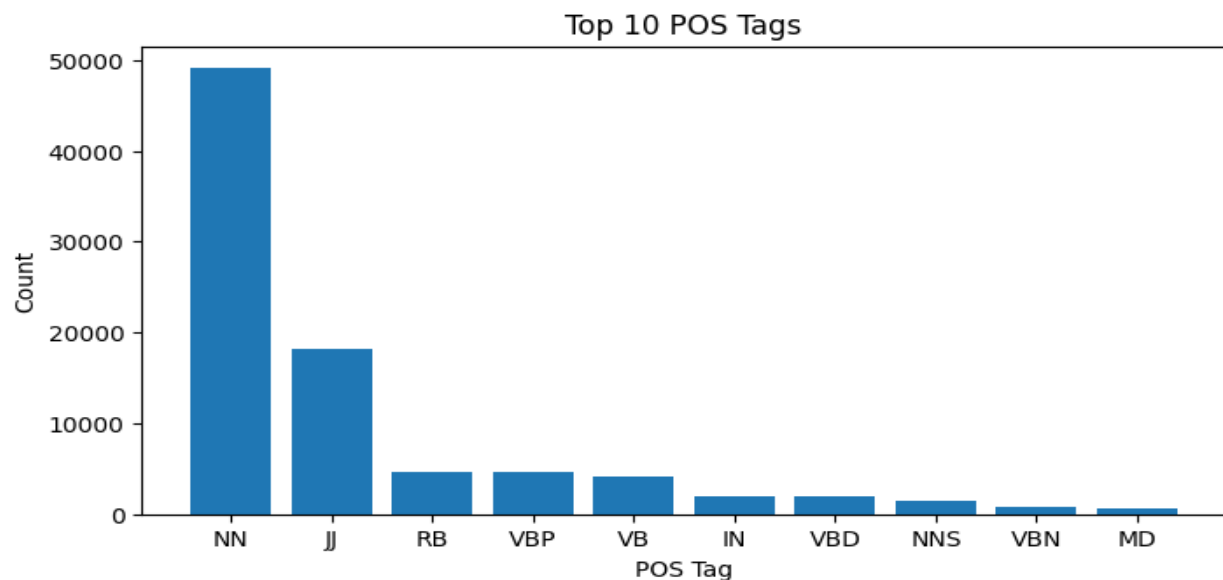


Figure 20 - Top 10 POS Visualization

4.5 Named Entity Recognition (NER)

Real world things that can be identified in text by Named Entity Recognition (NER) can include organizations, places and product names. NER can help to recognize companies, people, or locations that may frequently be mentioned in a review of e-commerce (Aya Data, 2024). NER was used to extract entities such as product brands and business names in the review text. The approach demonstrates that NLP may be applied to extract business-relevant information based on customer feedback along with sentiment classification.

```
# NER
import spacy

try:
    nlp = spacy.load("en_core_web_sm")
except:
    import spacy.cli
    spacy.cli.download("en_core_web_sm")
    nlp = spacy.load("en_core_web_sm")

# Run NER on a few samples
ner_samples = df["content"].dropna().sample(3, random_state=7).tolist()
for i, text in enumerate(ner_samples, 1):
    doc = nlp(text)
    ents = [(ent.text, ent.label_) for ent in doc.ents]
    print(f"\n--- NER Sample {i} ---")
    print(ents)

--- NER Sample 1 ---
[]

--- NER Sample 2 ---
[]

--- NER Sample 3 ---
[('7', 'CARDINAL'), ('LG G5', 'ORG'), ('LG G8', 'ORG'), ('9', 'CARDINAL'), ('midday', 'TIME'), ('Alexa', 'ORG')]

from spacy import displacy
doc = nlp("I bought an Apple iPhone in Bahrain on July 2025 and returned it back to Amazon.")
print([(ent.text, ent.label_) for ent in doc.ents])
displacy.render(doc, style="ent", jupyter=True)

[('Apple iPhone', 'ORG'), ('Bahrain', 'GPE'), ('July 2025', 'DATE'), ('Amazon', 'ORG')]
I bought an Apple iPhone ORG in Bahrain GPE on July 2025 DATE and returned it back to Amazon ORG .
```

Figure 21 - NER with example

Task 5 - Text Classification / Prediction

5.1 Task 5 Introduction

The objective of the task is to create and evaluate machine learning systems which can automatically label customer reviews into sentiment categories based on their text. The sentiment labels were predicted with the use of supervised learning methods where the text had to be preprocessed and numerically represented.

5.2 Sentiment Label Creation

As mentioned in task 2 the original dataset incorporates the numerical ratings of customers and below are the labels selected for this sentiment analysis project.

- **Negative:** Poor ratings (1-2 Stars)
- **Neutral:** Medium ratings (3 Stars)
- **Positive:** Excellent ratings (4-5 Stars)

```
# Labelling
def sentiment_label(score):
    if score <= 2:
        return 'Negative'
    elif score == 3:
        return 'Neutral'
    else:
        return 'Positive'

df['sentiment'] = df['score'].apply(sentiment_label)
df[['score', 'sentiment']].head()
```

	score	sentiment
0	5	Positive
1	1	Negative
2	5	Positive
3	2	Negative
4	5	Positive

Figure 22 - Sentiment Label Creation and testing

5.3 Train and Test Data Split

The data has been divided into training and testing sets to determine the ability of the models to generalize. Using the test set as a measure of the model in unseen reviews, the training set was used to derive patterns of the data. This approach provides a reliable extrapolation of the performance of the models in real cases and prevents overfitting.

```
# Train-Test Split
from sklearn.model_selection import train_test_split

X = df['final_text']
y = df['sentiment']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Figure 23 - Train and Test Data Split

5.4 Feature Extraction Using TF-IDF

The already processed text was converted into numeric feature vectors by using the Term Frequency-Inverse Document Frequency (TF-IDF). TF-IDF places more weight on words common to a specific text and less to the corpus.

TF-IDF was chosen due to:

- It deals with sparse text data of high dimensions.
- It reduces the impact of often used terms that are not very informative.
- It is effective with linear classifiers in situations that involve text classification.
- The resultant feature vectors were used as inputs in both classification models.

```
#TF-IDF
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(
    max_features=5000,
    ngram_range=(1,2)
)

X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)
```

Figure 24 - TF-IDF

5.5 Model Selection and Justification

Two machine learning models were chosen to be used in sentiment classification.

5.5.1 Multinomial Naive Bayes

Multinomial Naive Bayes is a popular text categorization problem probabilistic classifier.

This model was chosen because of:

- It is good in calculation.
- It gives a firm basis on sentiment analysis.
- It is compatible with large pool of texts.

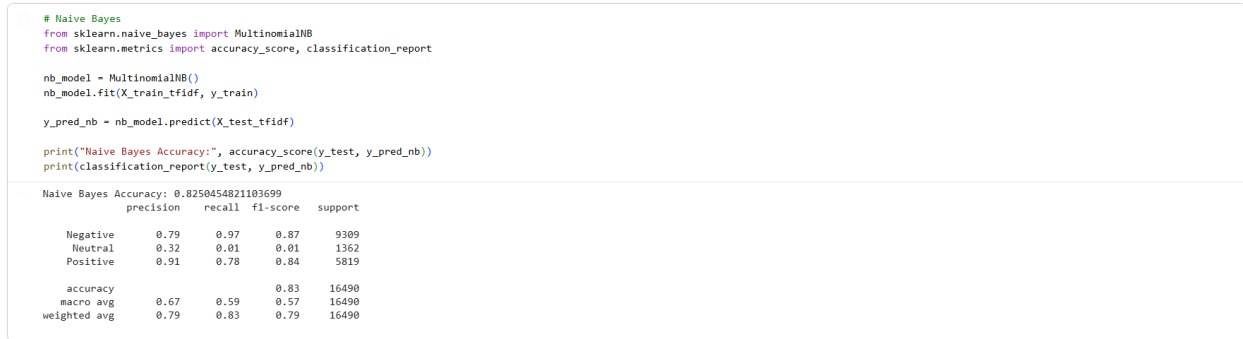


Figure 25 - Multinomial Naive Bayes

5.5.2 Logistic Regression

The logistic regression method of linear classification is rather effective in the high-dimensional feature space such as that produced by TF-IDF. It relies on a linear line of decisions to predict the possibility of membership to the class.

- The reason behind the choice of this model is that to:
- It can easily handle sparse text features.
- It is often more precise and recalls more than Naive Bayes.
- It provides consistent and understandable findings of sentiment categorization.

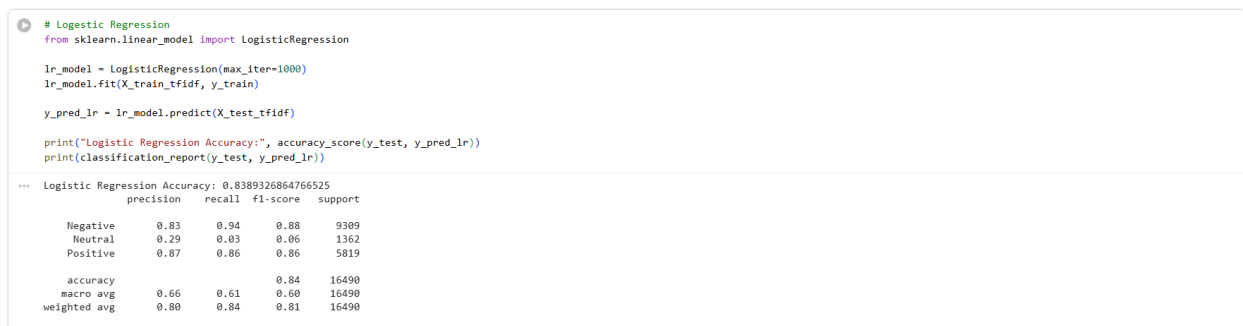


Figure 26 - Logistic Regression

5.6 Model Training and Prediction

Both models were trained using the feature representations of the training data in terms of TF-IDF. The test data were also predicted after training. Performance was then measured on the comparisons between the predicted sentiment labels and the actual labels.

Task 6 - Evaluation, Inferences, Recommendation and Reflection

6.1 Task 6 Introduction

The purpose of this task is to determine how effective sentiment categorization models were developed during the previous task. Evaluation measures are used to measure the effectiveness of the algorithms in forecasting the sentiment of the clients.

6.2 Evaluation Metrics

To measure the effectiveness of the categorization models, several evaluation metrics were used (Friedman, 2024):

- **Accuracy:** Measures the overall accuracy of the model.
- **Precision:** Indicates the percentage of sentiment labels that are predicted correctly.
- **Recall:** Determines the capability of the model to identify actual emotion classes.
- **F1-score:** Provides a reasonable evaluation of accuracy and memory.
- **Confusion Matrix:** This indicates the correct and incorrect prediction of each sentiment type.

Rather than relying on accuracy only, a range of indicators ensures that the model's performance is adequately evaluated.

```
# Calculate metrics
from sklearn.metrics import precision_recall_fscore_support

accuracy = accuracy_score(y_test, y_pred_lr)
precision, recall, f1, _ = precision_recall_fscore_support(
    y_test, y_pred_lr, average='weighted'
)

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1-Score:", f1)
```

```
... Accuracy: 0.8389326864766525
Precision: 0.7999575660111626
Recall: 0.8389326864766525
F1-Score: 0.808965286036539
```

Figure 27 - Evaluation Metrics

6.3 Model Performance Evaluation

The above metrics were employed in evaluating the Multinomial Naive Bayes and Logistic Regression models using the test data. The confusion matrices displayed the performance of each model in the three sentiment classes (Negative, Neutral and Positive). The results indicated that although neutral reviews were comparatively more challenging to sort out accurately, both models can classify positive and negative reviews quite successfully. Logistic Regression generally performed well on balance in terms of accuracy, recall and F1-score as compared to Multinomial Naive Bayes, suggesting that it was overall more successful.

6.4 Model Comparison

Logistic Regression was more successful particularly in terms of F1-score. Multinomial Naive Bayes is a baseline model and was not very good at processing sentiment expressions that were overlapping. Neutral sentiment reviews were harder to classify because of language ambiguity. TF-IDF with linear classifiers were effective in analyzing sentiments of customer ratings. These findings indicate that the correct model to use is crucial in achieving precise sentiment categorization results.

```

# Confusion Matrix Naive Bayes
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

cm_nb = confusion_matrix(y_test, y_pred_nb, labels=['Negative', 'Neutral', 'Positive'])

disp_nb = ConfusionMatrixDisplay(
    confusion_matrix=cm_nb,
    display_labels=['Negative', 'Neutral', 'Positive']
)

disp_nb.plot(cmap='Blues')
plt.title("Confusion Matrix - Naive Bayes")
plt.show()

```

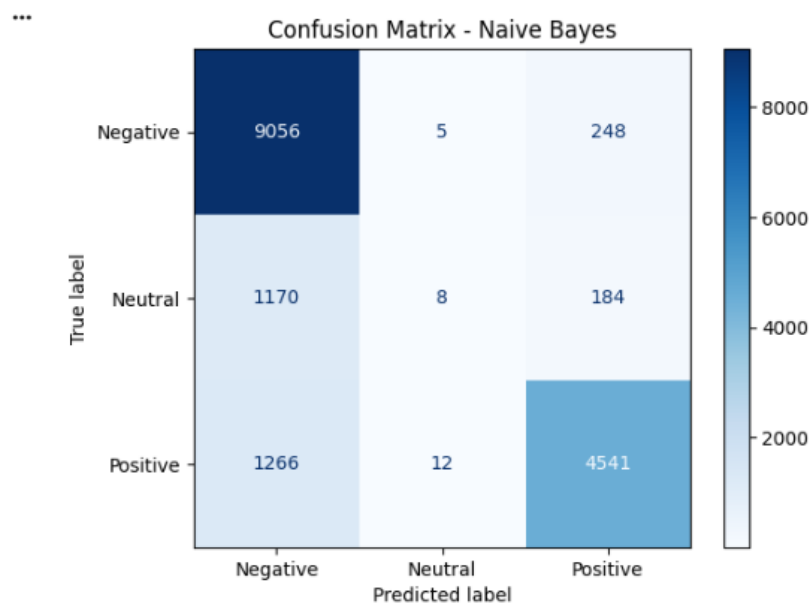


Figure 28 - Confusion Matrix Naive Bayes

```

# Confusion Martix Logistic Regression
cm_lr = confusion_matrix(y_test, y_pred_lr, labels=['Negative', 'Neutral', 'Positive'])

disp_lr = ConfusionMatrixDisplay(
    confusion_matrix=cm_lr,
    display_labels=['Negative', 'Neutral', 'Positive']
)

disp_lr.plot(cmap='Greens')
plt.title("Confusion Matrix - Logistic Regression")
plt.show()

```

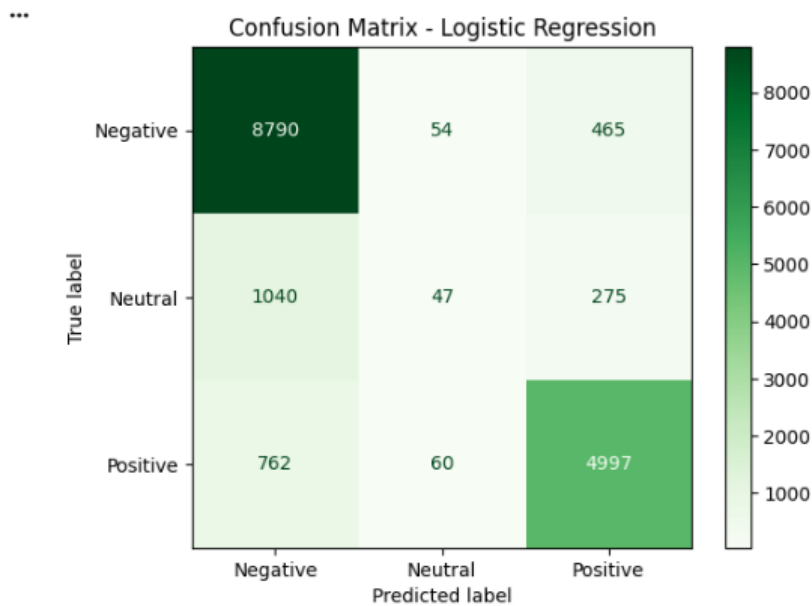


Figure 29 - Confusion Matrix Logistic Regression

6.7 Project Enhancements

Although the performance of the models is strong, additional studies could consider a series of improvements:

- Learning sequential relationships using deep learning such as LSTM.
- Transformer-based models like BERT are contextual understanding models.
- Resampling to solve the imbalance in classes.
- Installing sentiment lexicons and emojis instead of removing them.

6.8 Learning Reflection

The project provided me with the invaluable hands-on experience of natural language processing techniques on a real dataset. This project has allowed me to learn more about sentiment classification, feature extraction and text preprocessing. I also learned the impact of different types of classifiers and text representation on the performance of the model. Another benefit that the project brought me was the ability to critically evaluate models based on various indicators and interpret results in both technical and business perspectives. Overall, it is possible to state that this project enhanced my understanding of the principles of NLP and provided me with greater confidence in applying machine learning approaches to address real-world problems.

References

Aya Data. (2024, August 6). *What is named entity recognition in NLP?* AyaData.ai. <https://www.ayadata.ai/what-is-named-entity-recognition-in-nlp/>

Deepanshi. (2021, June 25). *Text preprocessing in NLP with Python codes*. Analytics Vidhya. <https://www.analyticsvidhya.com/blog/2021/06/text-preprocessing-in-nlp-with-python-codes/>

Friedman, Y. (2024, June 13). *Understanding F1 score, accuracy, ROC-AUC, and PR-AUC metrics for models*. Deepchecks. <https://www.deepchecks.com/f1-score-accuracy-roc-auc-and-pr-auc-metrics-for-models>

Kumar, A. (2025, December). *Amazon Shopping Reviews [Daily Updated]* [Dataset]. Kaggle. <https://www.kaggle.com/datasets/ashishkumarak/amazon-shopping-reviews-daily-updated>